

Reto QA - Fundamentos para Automatización

Ejercicio 1: Plan de Pruebas BDD | Ejercicio 2: TDD Fibonacci

Fecha: Septiembre 2025

EJERCICIO 1: PLAN DE PRUEBAS - FUNCIONALIDAD RESEÑAS DE PRODUCTOS

Contexto del Proyecto

Una startup va a lanzar una nueva funcionalidad crítica en su aplicación de e-commerce: "**Reseñas de Productos**". Los usuarios registrados podrán escribir una reseña (calificándola de 1 a 5 estrellas) y añadir un comentario de texto sobre los productos que han comprado.

1. Alcance de las Pruebas

Dentro del Alcance (In Scope)

- **Funcionalidad de escritura de reseñas:** Validar que los usuarios registrados puedan crear reseñas con calificación de 1 a 5 estrellas
- **Sistema de comentarios:** Verificar que se pueda añadir, editar y eliminar comentarios de texto en las reseñas
- **Validación de datos:** Comprobar validaciones de campos obligatorios, límites de caracteres y formato de calificación
- **Restricciones de acceso:** Confirmar que solo usuarios registrados y que han comprado el producto pueden escribir reseñas
- **Visualización de reseñas:** Verificar que las reseñas se muestren correctamente en la página del producto
- **Funcionalidad de edición/eliminación:** Validar que los usuarios puedan modificar o eliminar sus propias reseñas
- **Cálculo de promedio:** Verificar que el promedio de calificaciones se calcule y muestre correctamente

Fuera del Alcance (Out of Scope)

- **API de terceros para validación de compras:** No se probará la integración con sistemas externos de verificación de compras
- **Flujo de registro de nuevos usuarios:** El proceso de creación de cuentas no está incluido en este ciclo
- **Funcionalidades de pago:** Los procesos de compra y pago están fuera del alcance
- **Sistema de notificaciones por email:** Las notificaciones automáticas no se incluyen
- **Moderación automática de contenido:** Los sistemas de filtrado automático de comentarios inapropiados
- **Funcionalidades de redes sociales:** Compartir reseñas en redes sociales externas

2. Tipos de Pruebas a Aplicar

Pruebas Funcionales

Justificación: Esenciales para verificar que todas las funcionalidades básicas del sistema de reseñas funcionen según los requisitos especificados, incluyendo creación, edición, eliminación y visualización de reseñas.

Pruebas de Usabilidad

Justificación: Críticas para asegurar una experiencia de usuario intuitiva y satisfactoria, ya que la funcionalidad de reseñas impacta directamente en la confianza del usuario y las decisiones de compra.

Pruebas de Compatibilidad

Justificación: Necesarias para garantizar que la funcionalidad opere correctamente en diferentes navegadores, dispositivos y sistemas operativos, considerando la naturaleza multiplataforma del e-commerce.

Pruebas de Rendimiento

Justificación: Importantes para verificar que el sistema mantenga tiempos de respuesta aceptables al cargar y procesar reseñas, especialmente en productos con muchas reseñas.

3. Criterios de Salida

Criterio 1: Cobertura de Casos de Prueba

Descripción: Al menos el 95% de los casos de prueba planificados deben haber sido ejecutados y el 90% de ellos deben haber pasado exitosamente.

Medición: $(\text{Casos de prueba pasados} / \text{Total casos de prueba ejecutados}) \times 100$

Criterio 2: Defectos Críticos y Bloqueantes

Descripción: No debe haber defectos críticos o bloqueantes abiertos. Los defectos de severidad alta no deben exceder 2 casos abiertos.

Medición: Número de defectos críticos = 0, Número de defectos altos ≤ 2

4. Casos de Prueba en Gherkin (BDD)

Escenario 1: Camino Feliz - Usuario envía reseña válida

Feature: Escritura de reseñas de productos **Como** usuario registrado que ha comprado un producto **Quiero** escribir una reseña con calificación y comentario **Para** compartir mi experiencia con otros compradores **Scenario:** Usuario envía reseña completa y válida **Given** que soy un usuario registrado y logueado **And** he comprado el producto "Smartphone XYZ" **And** estoy en la página del producto "Smartphone XYZ" **When** hago clic en el botón "Escribir reseña" **And** selecciono una calificación de 4 estrellas **And** escribo el comentario "Excelente producto, muy buena calidad y rapidez de entrega" **And** hago clic en "Enviar reseña" **Then** debería ver el mensaje "Reseña enviada exitosamente" **And** mi reseña debería aparecer en la lista de reseñas del producto **And** la calificación promedio del producto debería actualizarse

Escenario 2: Escenario Negativo - Usuario intenta enviar reseña incompleta

Scenario: Usuario intenta enviar reseña sin calificación obligatoria **Given** que soy un usuario registrado y logueado **And** he comprado el producto "Laptop ABC" **And** estoy en la página del producto "Laptop ABC" **When** hago clic en el botón "Escribir reseña" **And** escribo el comentario "Producto decepcionante" **And** NO selecciono ninguna calificación de estrellas **And** hago clic en "Enviar reseña" **Then** debería ver el mensaje de error "La calificación es obligatoria" **And** la reseña NO debería ser enviada **And** debería permanecer en el formulario de reseña

Escenario 3: Caso Borde - Usuario envía comentario con longitud límite

Scenario: Usuario envía comentario con exactamente 500 caracteres (límite máximo) **Given** que soy un usuario registrado y logueado **And** he comprado el producto "Tablet DEF" **And** estoy en la página del producto "Tablet DEF" **When** hago clic en el botón "Escribir reseña" **And** selecciono una calificación de 3 estrellas **And** escribo un comentario de exactamente 500 caracteres **And** hago clic en "Enviar reseña" **Then** debería ver el mensaje "Reseña enviada exitosamente" **And** mi reseña debería aparecer completa en la lista de reseñas **And** el comentario debería mostrarse sin truncamiento

EJERCICIO 2: TDD PARA SECUENCIA FIBONACCI

Contexto del Proyecto

Se requiere desarrollar una función que genere la secuencia de Fibonacci hasta la secuencia 50 como una única cadena de texto. La función recibirá un número entero **n** que indica cuántos elementos de la secuencia se deben generar y concatenar.

Ejemplo esperado: Si se invoca `generarsecuenciafibonacci(7)`, el resultado debe ser la cadena de texto: "0112358"

Código Final de la Función

```
def generarsecuenciafibonacci(n): """ Genera la secuencia de Fibonacci hasta n
elementos como una cadena de texto. Args: n (int): Número de elementos de la secuencia
a generar Returns: str: Secuencia de Fibonacci concatenada como cadena """ if n <= 0:
return "" elif n == 1: return "0" elif n == 2: return "01" # Generar la secuencia
secuencia = [0, 1] for i in range(2, n): siguiente = secuencia[i-1] + secuencia[i-2]
secuencia.append(siguiente) # Convertir a cadena y concatenar return ''.join(map(str,
secuencia))
```

Pruebas Unitarias (TDD)

```
import unittest class TestFibonacciTDD(unittest.TestCase): """ Pruebas unitarias
desarrolladas siguiendo la metodología TDD. Cada test representa un ciclo RED -> GREEN
-> REFACTOR. """ def test_fibonacci_n_igual_1_retorna_0(self): """CICLO 1: Caso más
simple - n=1 debe retornar "0" """ resultado = generarsecuenciafibonacci(1)
self.assertEqual(resultado, "0") def test_fibonacci_n_igual_2_retorna_01(self):
"""CICLO 2: n=2 debe retornar "01" """ resultado = generarsecuenciafibonacci(2)
self.assertEqual(resultado, "01") def test_fibonacci_n_igual_3_retorna_011(self):
"""CICLO 3: n=3 debe retornar "011" """ resultado = generarsecuenciafibonacci(3)
self.assertEqual(resultado, "011") def test_fibonacci_n_igual_7_retorna_0112358(self):
"""CICLO 4: Ejemplo del enunciado - n=7 debe retornar "0112358" """ resultado =
generarsecuenciafibonacci(7) self.assertEqual(resultado, "0112358") def
test_fibonacci_n_igual_0_retorna_cadena_vacia(self): """CICLO 5: Caso borde - n=0 debe
retornar cadena vacía """ resultado = generarsecuenciafibonacci(0)
self.assertEqual(resultado, "") def
test_fibonacci_n_negativo_retorna_cadena_vacia(self): """CICLO 6: Caso borde - n
negativo debe retornar cadena vacía """ resultado = generarsecuenciafibonacci(-1)
self.assertEqual(resultado, "") def
test_fibonacci_n_igual_10_secuencia_completa(self): """CICLO 7: Prueba con secuencia
más larga - n=10 """ resultado = generarsecuenciafibonacci(10)
self.assertEqual(resultado, "01123581321") def
test_fibonacci_n_igual_15_incluye_numeros_grandes(self): """CICLO 8: Prueba con
números de múltiples dígitos - n=15 """ resultado = generarsecuenciafibonacci(15) # 0,
1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377 esperado =
"0112358132134558914423377" self.assertEqual(resultado, esperado)
```

Documentación del Proceso TDD

PROCESO TDD SEGUIDO PASO A PASO: CICLO 1 (RED → GREEN → REFACTOR): • RED: Escribí `test_fibonacci_n_igual_1_retorna_0()` esperando "0" • GREEN: Implementé la función retornando "0" cuando `n=1` • REFACTOR: No hubo necesidad de refactorizar en este punto

CICLO 2 (RED → GREEN → REFACTOR): • RED: Escribí `test_fibonacci_n_igual_2_retorna_01()` esperando "01" • GREEN: Agregué lógica para manejar `n=2` retornando "01" • REFACTOR: No hubo necesidad de refactorizar aún

CICLO 3 (RED → GREEN → REFACTOR): • RED: Escribí `test_fibonacci_n_igual_3_retorna_011()` esperando "011" • GREEN: Implementé lógica básica para generar secuencia con bucle • REFACTOR: Limpié el código organizando la lógica de generación

CICLO 4 (RED → GREEN → REFACTOR): • RED: Escribí test para `n=7` según el ejemplo del enunciado • GREEN: La implementación ya pasaba este test • REFACTOR: Optimicé la función para manejar casos más grandes eficientemente

CICLOS 5-8: • Continué agregando casos borde y escenarios más complejos • Refiné la función

asegurando robustez en todos los casos • Mantuve el código limpio y bien documentado
RESULTADO FINAL: ✓ Función robusta que maneja todos los casos de prueba ✓ Cobertura completa incluyendo casos borde ✓ Código limpio y bien documentado ✓ Pruebas exhaustivas que guiaron el desarrollo

Ejemplos de Ejecución

```
# Ejemplos de uso de la función: print(f"Fibonacci(1):  
{generarsecuenciafibonacci(1)}") # "0" print(f"Fibonacci(7):  
{generarsecuenciafibonacci(7)}") # "0112358" print(f"Fibonacci(10):  
{generarsecuenciafibonacci(10)}") # "01123581321" # Para ejecutar las pruebas: if  
__name__ == '__main__': unittest.main()
```

Conclusiones

La implementación TDD permitió desarrollar una función robusta y bien probada que cumple con todos los requisitos especificados. El proceso iterativo de RED-GREEN-REFACTOR aseguró que cada funcionalidad fuera correctamente implementada y verificada antes de continuar con el siguiente ciclo.

Las pruebas unitarias proporcionan cobertura completa de casos normales, casos borde y casos de error, garantizando la confiabilidad de la función en diferentes escenarios de uso.

Reto QA - Fundamentos para Automatización

Ejercicios completados siguiendo mejores prácticas de QA, BDD y TDD

Septiembre 2025